

## 06 — Glossary

Plain-English definitions of every technical term used in this project. Grouped by theme; skim for whatever you need.

### Ranking quality metrics

- **Relevance / graded relevance** — how good a candidate is for the JD. Here it's graded on tiers 0–5 (5 = ideal hire, 0 = irrelevant), not just yes/no.
- **NDCG@k (Normalised Discounted Cumulative Gain)** — the main quality metric. It rewards putting highly-relevant people near the top of the first  $k$  results. “Discounted” = lower positions count less; “Normalised” = divided by the best possible ordering, so it lands in  $[0, 1]$ . NDCG@10 looks at the top 10.
- **MAP (Mean Average Precision)** — averages how early the relevant candidates appear across the list. Rewards clustering good matches near the top.
- **P@10 (Precision at 10)** — the fraction of the top 10 that are actually relevant (here, tier  $\geq 3$ ).
- **Precision@k / Recall@k** — precision@k = of the  $k$  you returned, how many were relevant; recall@k = of all relevant items, how many you surfaced in the top  $k$ . Used to evaluate the graph search. Raising recall often lowers precision (you return more, catching more good ones but also more noise) — the **precision/recall trade-off**.
- **MRR (Mean Reciprocal Rank)** —  $1 /$  the rank of the first relevant result, averaged over queries. Rewards putting a good answer at the very top.
- **Composite score** — the contest's weighted blend:  $0.50 \cdot \text{NDCG@10} + 0.30 \cdot \text{NDCG@50} + 0.15 \cdot \text{MAP} + 0.05 \cdot \text{P@10}$ . The number you're optimising.

### Search & retrieval

- **BM25 (Okapi BM25)** — the standard keyword-relevance formula. Scores a document by how many query terms it contains, weighting **rarer** terms more and damping very long documents. “Sparse” retrieval (works on exact words).
- **TF-IDF / IDF** — term-frequency  $\times$  inverse-document-frequency. The idea BM25 builds on: common words count for little, rare words count for a lot.
- **Inverted index** — a map from each word to the list of documents containing it. The data structure that makes keyword search fast.
- **Embedding** — a list of numbers (a vector) representing the *meaning* of text, so that similar meanings sit close together. Here, 384 numbers per profile.
- **Cosine similarity** — measures the angle between two vectors;  $\sim 1$  = very similar,  $\sim 0$  = unrelated. How we compare an embedding to the JD's embedding. “Dense” retrieval (works on meaning, not exact words).
- **Bi-encoder** — encodes the query and each document *separately* into vectors, then compares. Fast (you can pre-compute all document vectors); used for first-stage retrieval. sentence-transformers' default mode.
- **Cross-encoder** — feeds the query and a document *together* through the model and outputs one relevance score. Much more accurate, much slower — so we only run it on the top  $\sim 300$  (the **reranker**).
- **RRF (Reciprocal Rank Fusion)** — a simple way to merge two ranked lists: add up  $1/(k + \text{rank})$  from each list. Combines BM25 and embeddings without needing their scores to be on the same scale.
- **Hybrid retrieval** — using keyword (sparse) and semantic (dense) search together, then fusing them. More robust than either alone.

### Machine learning

- **Feature / feature vector** — the fixed list of numbers describing one candidate (here, 40). Models only ever see these numbers.

- **GBDT (Gradient-Boosted Decision Trees)** — an ML model that combines many small decision trees, each correcting the last. XGBoost, LightGBM, and scikit-learn's HistGradientBoosting are all GBDTs. Great for tabular/feature data.
- **Learning-to-rank (LTR)** — training a model to *order* items, not to predict a number in isolation. Three flavours: **pointwise** (score each item), **pairwise** (which of two is better), **listwise** (optimise the whole list's metric).
- **LambdaMART** — a listwise LTR method: GBDTs trained to directly improve a ranking metric like NDCG. We use it via XGBoost's XGBRanker(objective="rank:ndcg").
- **Pseudo-labels** — labels you *create* when the dataset has none. We build graded relevance tiers from a trusted composite (and optionally an LLM) so the LTR model has something to learn from.
- **Proxy label / formula** — a hand-written relevance estimate from features we trust. Used to bootstrap training and as the ultimate fallback if no ML library is installed.
- **Bradley-Terry model** — a classic model for “who beats whom” data. Given the results of many pairwise matches, it estimates each player's **strength** so the strengths best explain the results — even resolving cycles ( $A > B > C > A$ ). We fit it with **MM iteration** (a stable, pure-NumPy maximum-likelihood algorithm).
- **MLE (Maximum Likelihood Estimation)** — pick the parameters that make the observed data most probable. How the Bradley-Terry strengths are found.
- **Normalisation (min-max)** — rescaling numbers into  $[0, 1]$  so different signals can be combined fairly.

## The two-phase design & performance

- **Two-phase split** — slow, smart work runs **offline** (precompute.py) and is cached; the **online** graded step (rank.py) only reads the cache, so it meets the no-network/CPU/<5-min limits.
- **Graceful degradation / fallback** — if a heavy dependency is missing, the code uses a simpler pure-Python path instead of crashing. The project's core design rule.
- **Content-hash cache** — store an artifact with a hash of its inputs; on a re-run, reuse it only if the hash still matches. Lets a JD change reuse the embeddings.
- **MPS / CUDA** — Apple-Silicon GPU / NVIDIA GPU. The transformers auto-detect and use them when present.
- **ONNX / quantization (int8)** — ONNX Runtime is a fast model-execution engine; quantization shrinks model weights to 8-bit integers for  $\sim 2\text{--}4\times$  CPU speed with tiny quality loss.

## Production service

- **FastAPI** — the Python web framework serving the HTTP API.
- **ORM (Object-Relational Mapping)** — write Python objects, the library (SQLAlchemy) turns them into SQL. db/models.py are ORM models.
- **Migration (Alembic)** — a versioned change to the database schema, so the DB can evolve safely over time.
- **Repository pattern** — all database access goes through one layer (db/repositories.py), so the rest of the code never writes raw SQL.
- **Strategy pattern** — pick one of several interchangeable implementations at runtime (e.g. XGBoost  $\rightarrow$  LightGBM  $\rightarrow$  sklearn  $\rightarrow$  formula). Keeps fallbacks clean.
- **JWT (JSON Web Token)** — a signed token proving who you are, sent on each request after login.
- **API key** — a long-lived secret a machine/script uses to authenticate (instead of logging in). Stored hashed; shown to you once.
- **RBAC (Role-Based Access Control)** — permissions by role (admin / recruiter / viewer).
- **bcrypt** — a deliberately slow password-hashing algorithm, so stolen hashes are hard to crack.
- **Redis** — a fast in-memory data store used here for caching and rate limiting.
- **Cache eviction (LRU / LFU)** — when the cache is full, drop the **Least Recently / Frequently Used** entries.
- **Rate limiting (fixed window)** — cap requests per time window to prevent abuse.
- **Message queue** — a buffer between “a request arrived” and “the work ran”, so the API can respond instantly and a worker processes jobs in the background.

- **Idempotent** — running the same operation twice has the same effect as once (so a duplicate job message is safely ignored).
- **At-least-once delivery** — the queue guarantees a message is processed at least once (retrying on failure); idempotency makes the possible duplicates harmless.
- **Dead-letter queue (DLQ)** — where messages go after too many failed retries, so they don't loop forever.
- **SSE (Server-Sent Events)** — a one-way stream from server to browser; used to push live job progress to the UI.
- **Circuit breaker** — stops calling a failing dependency for a while after repeated errors, then tests if it has recovered. Prevents cascading failure.
- **Prometheus / Grafana** — Prometheus collects metrics; Grafana charts them.
- **OpenTelemetry / Jaeger** — OpenTelemetry produces traces (the path of a request through the system); Jaeger displays them.
- **structlog** — logging library that emits structured JSON (machine-readable logs).
- **Celery** — runs background tasks on worker processes; “eager” mode runs them inline so no broker is needed in dev.

## The live signal ingestion layer

- **Proof-of-work graph** — a knowledge graph linking developers → skills → evidence (commits, PRs, issues). Each evidence node is **SHA-256 anchored** (content-hashed) so it's tamper-evident/verifiable.
- **Knowledge graph / Neo4j** — data stored as nodes and relationships; Neo4j is the graph database (with an in-memory fallback here).
- **Confidence score** — how much to trust a skill claim, from the evidence: source-weighted (a merged PR > a resume line), complexity-scaled, and **recency-decayed**.
- **Recency decay / half-life** — older evidence counts for less; a one-year half-life means evidence is worth half as much after a year.
- **Evidence bonus** — the additive term (default 0) by which graph-verified developers float slightly above identical static-only profiles. It augments the ranking, never overrides it.
- **GraphQL** — the query language GitHub's API uses; lets us fetch exactly the fields we need in one request.

## Project-specific terms

- **Honeypot** — a deliberately fake profile in the dataset (impossible career maths) planted as a trap; detected by strict rules and forced to score 0.
- **Gate / gating** — hard disqualifiers checked before ranking (e.g. consulting-only career, keyword stuffing, inactivity).
- **JobSpec** — the structured version of the job description (required skills, seniority, location) produced by the JD deconstructor.
- **Head (of the ranking)** — the top slice of candidates (~300 for rerank, ~60 for the tournament) where the score is actually decided.
- **Manifest** — `manifest.json`, the record of what a precompute run produced (hashes, counts, timings) used by `--resume`.

Back to the [index](#).